UNIVERSITATEA TEHNICĂ A MOLDOVEI

FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ

DEPARTAMENTUL INGINERIA SOFTWARE ȘI AUTOMATICĂ

Lucrare de laborator nr. 8

Tehnici de programare

GitHub pas cu pas

Unealta stricata reparata cu git

Realizat de : Bulat Madalina

st. gr. SI-262

Verificat de : Emil Cebotaros

asist. univ.

Chisinau 2026

1. SCOPUL LUCRĂRII

Scopul lucrării constă în însușirea principiilor de utilizare a sistemului de control al versiunilor Git și a platformei GitHub, precum și dezvoltarea abilităților de gestionare a depozitelor, salvare și transmitere a modificărilor prin comenzile specifice.

De asemenea, lucrarea urmărește dezvoltarea capacităților de identificare, analiză și corectare a erorilor din programele Python, utilizând Git pentru înregistrarea fiecărei modificări și păstrarea istoricului procesului de depanare.

2. REALIZAREA LUCRĂRII

2.1. Lucrarea A – GitHub, pas cu pas

A1. Crearea contului GitHub

Am accesat platforma GitHub și m-am autentificat în contul personal, pregătind mediul necesar pentru crearea și gestionarea depozitelor de proiecte.

A2. Instalarea și configurarea Git

Am verificat instalarea Git pe calculator folosind comanda git --version. Ulterior, am configurat numele utilizatorului și adresa de e-mail prin comenzile git config --global user.name și git config --global user.email. Am stabilit ramura principală cu numele main și am verificat configurațiile realizate.

A3. Crearea primului depozit

Am creat pe platforma GitHub un depozit public cu denumirea exercitiu-git. Am adăugat fișierul README.md, care conține informații despre proiect.

A4. Clonarea depozitului pe calculator

Am copiat adresa HTTPS a depozitului și am utilizat comanda git clone pentru a descărca proiectul pe calculator. Ulterior, am accesat folderul exercitiu-git și am verificat fișierele existente.

A5. Crearea primului fișier și realizarea unui commit

Am creat fișierul salut.py, în care am introdus instrucțiunea print("Salut, GitHub!"). Am executat programul și am verificat rezultatul în terminal. Apoi, am utilizat comenzile git status, git add și git commit pentru a înregistra prima modificare în istoricul depozitului.

A6. Transmiterea modificărilor pe GitHub

Am utilizat comanda git push pentru a transmite commit-ul realizat de pe calculator către depozitul GitHub. Am verificat ulterior în browser dacă fișierul salut.py a fost încărcat cu succes.

A7. Actualizarea depozitului prin git pull

Am modificat fișierul README.md direct pe platforma GitHub, adăugând un rând cu informații despre exercițiu. Ulterior, am utilizat comanda git pull pentru a aduce modificările pe calculator și am verificat conținutul actualizat al fișierului.

A8. Crearea fișierului .gitignore

Am creat un mediu virtual Python folosind comanda python -m venv .venv. Pentru a evita includerea fișierelor inutile în depozit, am creat fișierul .gitignore, în care am adăugat regulile .venv/ și __pycache__/. Am verificat cu git status că mediul virtual este ignorat de Git. Am salvat și transmis modificarea prin commit și push.

A9. Verificarea istoricului și a diferențelor

Am utilizat comanda git log --oneline pentru a consulta istoricul commit-urilor. Am modificat temporar fișierul salut.py și am verificat diferențele cu ajutorul comenzii git diff. Ulterior, am utilizat git restore salut.py pentru a anula modificarea și am verificat dacă depozitul a revenit la starea curată.

A10. Crearea celui de-al doilea depozit

Am creat pe calculator un folder nou cu denumirea exercitiu-2, în care am inițializat un depozit Git folosind comanda git init. Am creat fișierul README.md, am înregistrat prima modificare prin commit și am creat un depozit public cu aceeași denumire pe GitHub. Am conectat depozitul local la cel remote prin git remote add origin și am transmis fișierele folosind git push -u origin main.

A11. Verificarea finală

Am verificat ambele depozite pe platforma GitHub. În depozitul exercitiu-git am confirmat existența fișierelor README.md, salut.py și .gitignore, precum și a folderului depanare. În al doilea depozit, exercitiu-2, am verificat existența fișierului README.md și disponibilitatea depozitului public.

2.2. Lucrarea B – Unealta stricată, reparată cu Git

B1. Pregătirea mediului de lucru

Am pregătit folderul depanare în depozitul exercitiu-git, în care am introdus fișierele inventar.py și tinte.json. Am salvat starea inițială a programului printr-un commit cu mesajul „Pornire: inventar.py stricat”. Ulterior, am executat programul pentru a identifica erorile existente.

B2. Identificarea și corectarea erorilor

Am analizat mesajele de eroare afișate în terminal și am corectat succesiv problemele identificate în cod.

1. SyntaxError – incarca

Am corectat eroarea de sintaxă prin adăugarea celor două puncte lipsă la definirea structurii funcției incarca().

Commit: SyntaxError incarca: adaug doua puncte

2. IndentationError – port_valid

Am corectat indentarea instrucțiunii return None, astfel încât funcția să respecte structura corectă a blocurilor de cod.

Commit: IndentationError port_valid: indentez return None

3. NameError – tinta

Am corectat numele variabilei utilizate, înlocuind tinta cu tinte, pentru a corespunde numelui declarat în program.

Commit: NameError tinta: folosesc tinte

4. TypeError la afișare

Am corectat conversia valorii returnate de funcția len() în șir de caractere, folosind str(), pentru a evita combinarea incorectă a tipurilor de date.

Commit: TypeError la afisare: convertesc len la str

5. KeyError – serviciu

Am modificat accesarea datelor folosind metoda get(), pentru a evita apariția erorii atunci când cheia serviciului lipsește din datele JSON.

Commit: KeyError serviciu: folosesc get cu rezerva

6. NoneType – pe_servicii

Am corectat funcția pentru a returna rezultatul grupării serviciilor, prin adăugarea instrucțiunii return grupe.

Commit: NoneType pe_servicii: return grupe

7. Lista serviciilor

Am corectat logica listei de servicii, astfel încât aceasta să nu mai fie resetată la fiecare repetare a buclei și să păstreze toate elementele identificate.

Commit: Lista servicii: nu resetez lista existenta

8. TypeError la prag

Am modificat configurarea argumentului --prag din argparse, adăugând type=int, pentru ca valoarea introdusă în terminal să fie interpretată ca număr întreg.

Commit: TypeError la prag: type=int in argparse

9. ValueError la port

Am utilizat funcția port_valid pentru verificarea porturilor și pentru omiterea valorilor invalide, fără întreruperea executării programului.

Commit: ValueError port: folosesc port_valid si sar peste invalid

10. Corectarea rezultatului incomplet

Am identificat o problemă în logica funcției, cauzată de poziționarea incorectă a instrucțiunii return. Am mutat instrucțiunea după buclă, astfel încât funcția să prelucreze toate elementele înainte de returnarea

rezultatului.

Commit: Rezultat incomplet: mut return dupa bucla

După fiecare corectare am înregistrat modificarea printr-un commit separat, utilizând Git pentru păstrarea istoricului procesului de depanare.

B3. Verificarea finală și transmiterea modificărilor

După corectarea tuturor erorilor, am executat programul cu pragul de 1000 și apoi fără specificarea pragului, pentru a verifica și valoarea implicită de 1024. Am verificat rezultatele afișate și generarea fișierului raport_inventar.json.

La final, am înregistrat rezultatul prin commit-ul „Raportul uneltei reparate” și am utilizat comenzile git push și git log --oneline pentru transmiterea modificărilor pe GitHub și verificarea istoricului.

3. REZULTATELE OBȚINUTE

În urma realizării lucrării am creat și configurat două depozite GitHub publice:

- exercitiu-git: https://github.com/madalinabulat72-droid/exercitiu-git
- exercitiu-2: https://github.com/madalinabulat72-droid/exercitiu-2

În primul depozit am verificat existența fișierelor README.md, salut.py, .gitignore și a folderului depanare. Acesta conține fișierele inventar.py, tinte.json și raport_inventar.json.

În cadrul lucrării B am corectat zece erori ale programului Python, fiecare reparație fiind înregistrată printr-un commit separat. Am verificat istoricul cu ajutorul comenzii git log --oneline și am confirmat transmiterea modificărilor pe GitHub.

4. CONCLUZII

În urma realizării Laboratorului 8 am dobândit cunoștințe practice despre utilizarea sistemului Git și a platformei GitHub pentru gestionarea proiectelor și păstrarea istoricului modificărilor.

Am învățat să creez și să gestionez depozite, să utilizez comenzile git init, git clone, git add, git commit, git push, git pull, git log, git diff și git restore.

De asemenea, prin repararea programului inventar.py, mi-am dezvoltat capacitatea de a identifica, analiza și corecta erorile de sintaxă, de tip, de nume și de logică dintr-un program Python. Am înțeles importanța testării după fiecare modificare și a înregistrării separate a reparațiilor prin commit-uri.

Astfel, scopul lucrării a fost realizat prin aplicarea practică a comenzilor Git, gestionarea depozitelor GitHub și depanarea programului Python, obținând un proiect funcțional și un istoric clar al modificărilor efectuate.